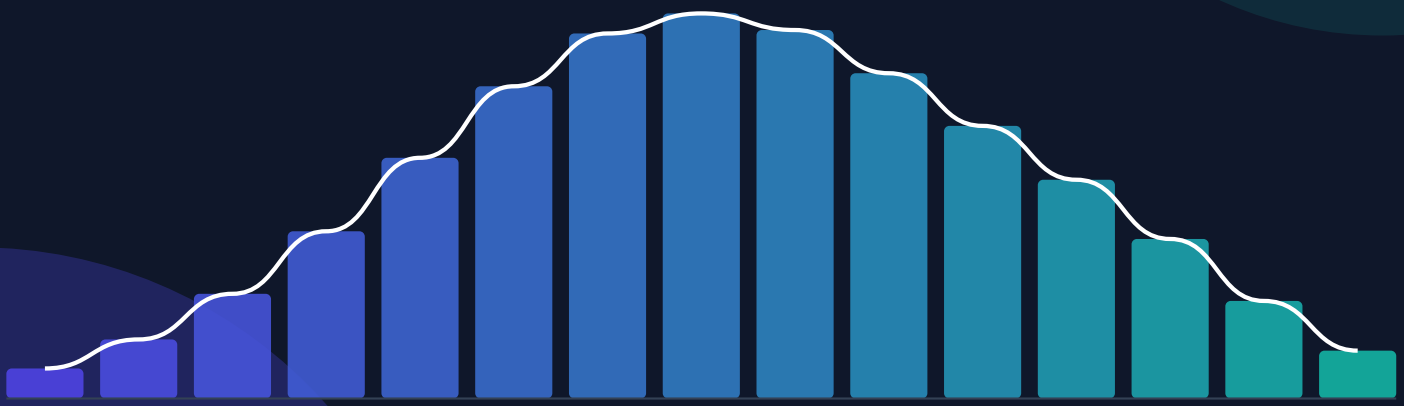
ПРАКТИЧЕСКОЕ РУКОВОДСТВО

Первичный анализ данных (EDA) в машинном обучении

Пошаговый гайд с готовым кодом на Python:
от первой загрузки данных до чек-листа перед моделью



11
этапов EDA

38
блоков кода

36 из 38
блоков проверено запуском

pandas · numpy · scipy · scikit-learn · seaborn · matplotlib

Содержание

0. Подготовка окружения . . . 3
- 0.1 Установка и импорты . . . 3
- 0.2 Загрузка и ранний split . . . 4

1. Знакомство с данными . . . 5
- 1.1 Первый взгляд . . . 5
- 1.2 Типы признаков и сводная таблица . . . 5
- 1.3 Оптимизация памяти . . . 6

2. Целевая переменная . . . 7
- 2.1 Классификация: баланс классов . . . 7
- 2.2 Регрессия: форма распределения . . . 7

3. Качество данных . . . 8
- 3.1 Пропуски . . . 8
- 3.2 Дубликаты . . . 9
- 3.3 Некорректные значения . . . 9
- 3.4 Несогласованность категорий . . . 9
- 3.5 Константные признаки и идентификаторы . . . 10

4. Одномерный анализ . . . 11
- 4.1 Числовые признаки . . . 11
- 4.2 Категориальные признаки . . . 12
- 4.3 Временные признаки . . . 12

5. Выбросы и аномалии . . . 13
- 5.1 IQR, z-score и робастный z-score . . . 13
- 5.2 Многомерные аномалии: Isolation Forest . . . 14
- 5.3 Что делать с выбросами . . . 14

6. Многомерный анализ . . . 15
- 6.1 Корреляции числовых признаков . . . 15
- 6.2 Категориальные признаки: Cramér's V . . . 15
- 6.3 Связь признаков с целевой переменной . . . 16
- 6.4 Мультиколлинеарность: VIF . . . 16
- 6.5 Парные графики . . . 17

7. Распределения и гипотезы . . . 18
- 7.1 Проверка нормальности . . . 18
- 7.2 Сравнение групп и связь с целью . . . 18
- 7.3 Множественные сравнения . . . 19

8. Проверка на утечки данных . . . 20
- 8.1 Подозрительно сильные признаки . . . 20
- 8.2 Пересечения train / test . . . 20
- 8.3 Корректность временного разбиения . . . 21

9. Репрезентативность и смещения . . . 21
- 9.1 Сравнение распределений train и test: KS-тест и PSI . . . 21
- 9.2 Adversarial validation . . . 22
- 9.3 Дрейф во времени . . . 22
- 9.4 Группы, смещения и чувствительные признаки . . . 22

10. Идеи по подготовке данных . . . 23
- 10.1 Пайплайн предобработки . . . 23
- 10.2 Идеи для новых признаков . . . 24
- 10.3 Валидация и быстрый бейзлайн . . . 24

11. Документирование результатов . . . 25
- 11.1 Автоматические отчёты . . . 25
- 11.2 Структура итогового отчёта . . . 25
- 11.3 Воспроизводимость . . . 25

A. Ошибки и чек-лист . . . 27
- A.1 Типичные ошибки . . . 27
- A.2 Итоговый чек-лист . . . 27

0 Подготовка окружения

Импорты, ранний split и соглашения, на которых держится весь гайд

Во всех примерах ниже используются одни и те же имена переменных. Если вы подставите свои данные в блок 0.2, остальной код будет работать почти без правок. Код рассчитан на Jupyter: функция `display()` есть там по умолчанию, в обычном скрипте замените её на `print()`. Блоки идут последовательно: поздние используют переменные из ранних.

Переменная	Что это
`df_raw`	исходная таблица целиком (до любых решений по данным)
`train`, `test`	результат раннего разбиения; `test` на этапе EDA не трогаем
`df`	рабочая копия `train`, на ней проводим весь EDA
`TARGET`	имя целевой переменной (в примерах бинарная 0/1)
`ID_COLS`	идентификаторы, которые не являются признаками
`num_cols`, `cat_cols`	списки числовых и категориальных признаков

0.1 Установка и импорты

Код 0.1 — зависимости и общие настройки PYTHON

```python
# pip install pandas numpy scipy scikit-learn matplotlib seaborn
import warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats

warnings.filterwarnings("ignore")
pd.set_option("display.max_columns", 100)
pd.set_option("display.float_format", "{:.3f}".format)
sns.set_theme(style="whitegrid", context="notebook")
plt.rcParams["figure.dpi"] = 110

SEED = 42
TARGET = "target"
ID_COLS = ["user_id"]
```

0.2 Загрузка и ранний split

Любое решение, которое «учится» на данных (импутация, масштабирование, отбор признаков, границы выбросов), должно опираться только на обучающую часть. Поэтому разбиваем данные **до** анализа и дальше работаем только с `train`.

Код 0.2 — загрузка и разбиение PYTHON

```python
from sklearn.model_selection import train_test_split

df_raw = pd.read_csv(
    "data.csv",
    sep=",",
    encoding="utf-8",
    parse_dates=["date"],      # столбцы с датами сразу в datetime
    low_memory=False,
)

train, test = train_test_split(
    df_raw,
    test_size=0.2,
    random_state=SEED,
    stratify=df_raw[TARGET],   # для классификации; для регрессии уберите
)

df = train.copy()              # EDA делаем только на этой копии
print(train.shape, test.shape)
```

ВНИМАНИЕ

Если данные имеют временную структуру (прогноз будущего), случайный split даст завышенную оценку качества. Делите по времени: `test` должен идти **после** `train` по дате (см. раздел 8.3).

1 Знакомство с данными

Размер, типы, смысл признаков, память

Цель этапа: убедиться, что данные прочитаны корректно, и понять, из чего они состоят. Ошибки чтения (съехавшие столбцы, числа как строки, неверная кодировка) гораздо дешевле найти здесь, чем после обучения модели.

1.1 Первый взгляд

Код 1.1 — размер, примеры строк, info — PYTHON

```python
print("Строк:", df.shape[0], "| Столбцов:", df.shape[1])

display(df.head())                    # начало таблицы
display(df.tail())                    # конец (часто видны артефакты выгрузки)
display(df.sample(5, random_state=SEED))   # случайные строки

df.info(memory_usage="deep")          # типы, число непустых, память
display(df.describe(include="all").T) # общая статистика по всем столбцам
```

СОВЕТ

Смотрите не только на `head()`: данные часто отсортированы, и первые строки нетипичны. `sample()` даёт более честное представление.

1.2 Типы признаков и сводная таблица

Pandas определяет тип автоматически и часто ошибается: числа с запятой читаются как строки, коды регионов как числа. Составьте сводную таблицу и проверьте её глазами, а затем разнесите признаки по группам.

Код 1.2 — типы и группы признаков — PYTHON

```python
overview = pd.DataFrame({
    "dtype": df.dtypes.astype(str),
    "n_unique": df.nunique(),
    "missing_%": (df.isna().mean() * 100).round(2),
    "example": df.iloc[0],
})
display(overview)

# Группы признаков (идентификаторы и цель исключаем)
drop_from_features = [TARGET] + ID_COLS
num_cols = (
    df.select_dtypes(include="number")
    .columns.difference(drop_from_features)
    .tolist()
)
dt_cols = df.select_dtypes(include="datetime").columns.tolist()
cat_cols = (
    df.select_dtypes(exclude=["number", "datetime", "timedelta"])
    .columns.difference(drop_from_features)
```

```
    .tolist()
)
print("Числовые:", num_cols)
print("Категориальные:", cat_cols)
print("Даты:", dt_cols)

# Числовые с малым числом значений — вероятно, категории
few_values = [c for c in num_cols if df[c].nunique() <= 10]
print("Числовые, похожие на категории:", few_values)
```

ЗАМЕТКА

Составьте словарь данных: что означает каждый столбец, в каких единицах он измерен, когда и как появляется. Если смысл признака непонятен, спросите эксперта предметной области: это самый частый источник скрытых утечек (раздел 8).

1.3 Оптимизация памяти

Для больших таблиц имеет смысл понизить разрядность чисел и перевести строки с малым числом значений в `category`. Это ускоряет работу и экономит гигабайты.

Код 1.3 — уменьшение занимаемой памяти PYTHON

```
def reduce_memory(frame, cat_threshold=0.5):
    out = frame.copy()
    for c in out.columns:
        s = out[c]
        if pd.api.types.is_integer_dtype(s):
            out[c] = pd.to_numeric(s, downcast="integer")
        elif pd.api.types.is_float_dtype(s):
            out[c] = pd.to_numeric(s, downcast="float")   # float32: меньше точность
        elif pd.api.types.is_string_dtype(s) or s.dtype == "object":
            if s.nunique() / max(len(s), 1) < cat_threshold:
                out[c] = s.astype("category")
    return out

before = df.memory_usage(deep=True).sum() / 1024**2
df_small = reduce_memory(df)
after = df_small.memory_usage(deep=True).sum() / 1024**2
print(f"Память: {before:.2f} МБ -> {after:.2f} МБ")
```

2 Целевая переменная

Тип задачи, баланс классов, форма распределения

Цель определяет всё: метрику, схему валидации, нужные преобразования. Изучайте её первой и внимательнее всего.

> **ЗАМЕТКА**
> Ниже разобраны классификация и регрессия. Для ранжирования смотрите на размеры групп и распределение релевантности, а в кластеризации целевой переменной нет: там вместо этого изучается структура самих признаков (разделы 4–6).

2.1 Классификация: баланс классов

Код 2.1 — распределение классов PYTHON

```python
counts = df[TARGET].value_counts()
share = df[TARGET].value_counts(normalize=True)
print(pd.DataFrame({"count": counts, "share": share.round(3)}))
print("Imbalance ratio (мажоритарный / миноритарный):",
      round(share.max() / share.min(), 2))

sns.countplot(x=TARGET, data=df)
plt.title("Распределение целевой переменной")
plt.show()
```

Доля миноритарного класса	Что делать
30–50 %	ситуация нормальная, достаточно stratified-валидации
5–30 %	метрики PR-AUC / F1, `class_weight`, порог подбирать отдельно
менее 5 %	то же + рассмотреть сэмплирование, anomaly detection, стратегию сбора данных

2.2 Регрессия: форма распределения

Для регрессии смотрим на асимметрию, «тяжёлые» хвосты и выбросы. Если цель сильно скошена вправо (цены, доходы, время), логарифмирование часто заметно улучшает модель.

Код 2.2 — гистограмма, логарифм, QQ-plot PYTHON

```python
y = df[TARGET]
print(y.describe(percentiles=[.01, .25, .5, .75, .99]))
print("Асимметрия:", round(y.skew(), 3), "| эксцесс:", round(y.kurt(), 3))

fig, ax = plt.subplots(1, 3, figsize=(15, 4))
sns.histplot(y, kde=True, ax=ax[0]);                ax[0].set_title("target")
sns.histplot(np.log1p(y.clip(lower=0)), kde=True, ax=ax[1])
ax[1].set_title("log1p(target)")
```

```
stats.probplot(y, dist="norm", plot=ax[2]);          ax[2].set_title("QQ-plot")
plt.tight_layout()
plt.show()
```

3 Качество данных

Пропуски, дубликаты, невозможные значения, константы

Здесь мы ищем всё, что может испортить обучение: пропуски, повторы, ошибки ввода, несогласованные форматы. Цель этапа: составить список проблем и решить, как с каждой поступать.

3.1 Пропуски

Важно не только сколько пропусков, но и **почему** они возникли. От механизма зависит выбор метода заполнения.

Механизм	Суть	Пример
MCAR	пропуск не связан ни с чем	сбой датчика
MAR	зависит от других наблюдаемых признаков	пожилые чаще не указывают email
MNAR	зависит от самого пропущенного значения	высокий доход чаще скрывают

Код 3.1 — доля, совместные пропуски, связь с целью PYTHON

```python
# 1) Доля пропусков по признакам
miss = df.isna().mean().sort_values(ascending=False)
miss = miss[miss > 0]
print((miss * 100).round(2))
if len(miss):
    miss.mul(100).plot.barh(figsize=(6, 0.4 * len(miss) + 1))
    plt.xlabel("% пропусков")
    plt.show()

# 2) Совместные пропуски: пропадают ли признаки вместе?
cols_na = df.columns[df.isna().any()]
if len(cols_na) > 1:
    sns.heatmap(df[cols_na].isna().corr(), annot=True, fmt=".2f",
                cmap="coolwarm", center=0)
    plt.title("Корреляция индикаторов пропусков")
    plt.show()

# 3) Связан ли факт пропуска с целью? (бинарная цель 0/1)
for c in cols_na:
    rate = df.groupby(df[c].isna())[TARGET].mean()
    print(f"{c}: доля target=1 при пропуске {rate.get(True, float('nan')):.3f}, "
          f"без пропуска {rate.get(False, float('nan')):.3f}")
```

СОВЕТ
Если доля target сильно различается при наличии и отсутствии значения, сам факт пропуска информативен. Добавьте индикатор `is_missing` (см. раздел 10.1), а не просто заполняйте медианой.

3.2 Дубликаты

Код 3.2 — полные и частичные дубликаты PYTHON

```python
print("Полных дубликатов:", df.duplicated().sum())

# Частичные: одинаковые значения ключевых полей
key = ["user_id", "date"]
dup_mask = df.duplicated(subset=key, keep=False)
print("Строк с повторяющимся ключом:", dup_mask.sum())
display(df[dup_mask].sort_values(key).head(10))

# Дубликаты с противоречивой целью — особенно опасны
conflict = (
    df[dup_mask].groupby(key)[TARGET].nunique().loc[lambda s: s > 1]
)
print("Ключей с разной целевой переменной:", len(conflict))
```

3.3 Некорректные значения

Опишите бизнес-правила в виде проверок. Это быстрее и надёжнее, чем искать глазами.

Код 3.3 — проверка бизнес-правил PYTHON

```python
rules = {
    "возраст вне [0, 110]":   (df["age"] < 0) | (df["age"] > 110),
    "дата из будущего":       df["date"] > pd.Timestamp.today(),
    "отрицательный доход":    df["income"] < 0,
}
report = pd.Series({name: int(mask.sum()) for name, mask in rules.items()})
print(report)

bad_any = np.logical_or.reduce([m.to_numpy() for m in rules.values()])
print(f"Строк с нарушениями: {bad_any.sum()} ({bad_any.mean():.2%})")
display(df.loc[bad_any].head())
```

3.4 Несогласованность категорий

Код 3.4 — разный регистр, пробелы, опечатки PYTHON

```python
for c in cat_cols:
    cleaned = df[c].astype("string").str.strip().str.lower()
    before, after = df[c].nunique(), cleaned.nunique()
    if after < before:
        print(f"{c}: {before} -> {after} уникальных после нормализации")
        display(df[c].value_counts().head(10))

# Поиск «почти одинаковых» значений (опечатки)
```

```python
from difflib import get_close_matches

def similar_values(series, cutoff=0.85):
    values = series.dropna().astype(str).unique().tolist()
    return {v: get_close_matches(v, [x for x in values if x != v], n=3, cutoff=cutoff)
            for v in values}

for c in cat_cols:
    close = {k: v for k, v in similar_values(df[c]).items() if v}
    if close:
        print(c, "->", list(close.items())[:5])
```

3.5 Константные признаки и идентификаторы

Код 3.5 — константы, почти константы, ID PYTHON

```python
nun = df.nunique(dropna=False)
const_cols = nun[nun <= 1].index.tolist()

top_share = {c: df[c].value_counts(normalize=True, dropna=False).iloc[0]
             for c in df.columns}
near_const = [c for c, s in top_share.items() if 0.99 < s < 1]

high_card = [c for c in cat_cols if df[c].nunique() / len(df) > 0.5]
unique_cols = [c for c in df.columns if df[c].is_unique]   # кандидаты в ID

print("Константы:", const_cols)
print("Почти константы (>99% одного значения):", near_const)
print("Высокая кардинальность:", high_card)
print("Полностью уникальные (похожи на ID):", unique_cols)
```

ВНИМАНИЕ

Почти константный признак не всегда бесполезен: редкое значение может идеально предсказывать редкий класс. Проверяйте связь с целью, прежде чем удалять.

4 Одномерный анализ

Распределение каждого признака по отдельности

4.1 Числовые признаки

Код 4.1 — статистики и графики PYTHON

```python
desc = df[num_cols].describe(percentiles=[.01, .05, .25, .5, .75, .95, .99]).T
desc["skew"] = df[num_cols].skew()
desc["kurt"] = df[num_cols].kurt()
desc["cv"] = desc["std"] / desc["mean"].abs()      # коэффициент вариации
display(desc.round(3))


def plot_numeric(frame, cols, kind="hist", ncols=3):
    nrows = int(np.ceil(len(cols) / ncols))
    fig, axes = plt.subplots(nrows, ncols, figsize=(5 * ncols, 3.4 * nrows))
    axes = np.atleast_1d(axes).ravel()
    for ax, c in zip(axes, cols):
        if kind == "hist":
            sns.histplot(frame[c], kde=True, ax=ax)
        else:
            sns.boxplot(x=frame[c], ax=ax)
        ax.set_title(c)
    for ax in axes[len(cols):]:
        ax.axis("off")
    plt.tight_layout()
    plt.show()


plot_numeric(df, num_cols, "hist")
plot_numeric(df, num_cols, "box")
```

Что видим	Что это значит	Что делать
|skew| > 1	сильная асимметрия	log1p, Box-Cox, квантильное преобразование
пик в одном значении	заглушка (0, -1, 9999)	отдельный индикатор + замена на NaN
несколько горбов	смесь подгрупп	искать признак-разделитель, кластеризация
тяжёлые хвосты	выбросы или реальная природа	см. раздел 5

4.2 Категориальные признаки

Код 4.2 — частоты и редкие категории PYTHON

```python
for c in cat_cols:
    vc = df[c].value_counts(dropna=False)
    rare = vc[vc / len(df) < 0.01]
    print(f"\n=== {c}: {len(vc)} уникальных, редких (<1%): {len(rare)} ===")
    print(vc.head(10))

# Визуализация первых категориальных признаков (топ-10 значений)
fig, axes = plt.subplots(1, min(3, len(cat_cols)), figsize=(15, 3.6))
for ax, c in zip(np.atleast_1d(axes), cat_cols):
    df[c].value_counts().head(10).sort_values().plot.barh(ax=ax)
    ax.set_title(c)
plt.tight_layout()
plt.show()
```

4.3 Временные признаки

Код 4.3 — диапазон, пропуски в ряду, сезонность PYTHON

```python
ts = df.sort_values("date").set_index("date")
print("Диапазон:", ts.index.min(), "->", ts.index.max())

# Число записей по дням и пропущенные дни
daily = ts.resample("D").size()
print("Дней без записей:", int((daily == 0).sum()), "из", len(daily))
daily.rolling(7).mean().plot(figsize=(11, 3), title="Записей в день (скользящее среднее 7)")
plt.show()

# Тренд и сезонность цели
fig, ax = plt.subplots(1, 2, figsize=(12, 3.5))
ts[TARGET].resample("MS").mean().plot(ax=ax[0], marker="o", title="Среднее target по месяцам")
df.groupby(df["date"].dt.dayofweek)[TARGET].mean().plot.bar(
    ax=ax[1], title="Среднее target по дню недели")
plt.tight_layout()
plt.show()
```

5 Выбросы и аномалии

Обнаружение, диагностика природы, решение

Выброс выбросу рознь. Это может быть ошибка ввода (удалить или исправить), редкое, но реальное событие (оставить, возможно ограничить влияние) или сигнал о важной подгруппе (изучить отдельно). Сначала понимаем природу, и только потом решаем.

Метод	Когда подходит	Ограничение
IQR (1.5 × IQR)	почти любые одномерные данные	на скошенных данных ловит много «лишнего»
z-score (|z| > 3)	примерно нормальные признаки	сам чувствителен к выбросам
Robust z (MAD)	скошенные данные, малые выборки	при MAD = 0 не работает
Isolation Forest	многомерные аномалии	нужно задавать contamination

5.1 IQR, z-score и робастный z-score

Код 5.1 — одномерные методы PYTHON

```python
def iqr_outliers(s, k=1.5):
    q1, q3 = s.quantile([0.25, 0.75])
    iqr = q3 - q1
    return (s < q1 - k * iqr) | (s > q3 + k * iqr)


def robust_z(s):
    med = s.median()
    mad = (s - med).abs().median()
    return 0.6745 * (s - med) / mad


rows = {}
for c in num_cols:
    s = df[c].dropna()
    rows[c] = {
        "iqr_%":      100 * iqr_outliers(s).mean(),
        "z>3_%":      100 * (np.abs(stats.zscore(s)) > 3).mean(),
        "robust_z_%": 100 * (robust_z(s).abs() > 3.5).mean(),
    }
outlier_report = pd.DataFrame(rows).T.sort_values("iqr_%", ascending=False)
display(outlier_report.round(2))

# Посмотреть сами строки для самого «проблемного» признака
worst = outlier_report.index[0]
mask = iqr_outliers(df[worst].dropna())
display(df.loc[mask[mask].index].sort_values(worst).head(10))
```

5.2 Многомерные аномалии: Isolation Forest

Строка может выглядеть нормально по каждому признаку отдельно, но быть странной в сочетании (возраст 5 лет и доход 200 тыс.). Такие случаи ловят многомерные методы.

Код 5.2 — Isolation Forest PYTHON

```python
from sklearn.ensemble import IsolationForest

X_if = df[num_cols].fillna(df[num_cols].median())
iso = IsolationForest(n_estimators=300, contamination=0.01, random_state=SEED)
iso.fit(X_if)

df["anomaly_score"] = -iso.score_samples(X_if)     # больше = аномальнее
df["is_anomaly"] = iso.predict(X_if) == -1

print("Аномалий:", int(df["is_anomaly"].sum()))
display(df.sort_values("anomaly_score", ascending=False).head(10))

# Отличается ли доля целевого класса среди аномалий?
print(df.groupby("is_anomaly")[TARGET].agg(["mean", "count"]))
```

5.3 Что делать с выбросами

Код 5.3 — winsorization и преобразования PYTHON

```python
# Ограничение (winsorization): границы считаем ТОЛЬКО по train
low, high = df["income"].quantile([0.01, 0.99])
df["income_w"] = df["income"].clip(low, high)
test_income_w = test["income"].clip(low, high)     # те же границы для test

# Логарифм для скошенного признака
df["income_log"] = np.log1p(df["income"].clip(lower=0))

# Сравнение асимметрии до и после
print(df[["income", "income_w", "income_log"]].skew().round(2))
```

ВНИМАНИЕ

Удаляйте строки с выбросами только если уверены, что это ошибка. Для деревьев выбросы обычно безвредны; для линейных моделей и нейросетей критичны. Решение зависит от модели.

6 Многомерный анализ

Связи между признаками и с целью

Пара признаков	Мера связи	Инструмент
числовой – числовой	Пирсон (линейная), Спирмен (монотонная)	`df.corr()`
категориальный – категориальный	Cramér's V, χ^2	функция ниже
числовой – категориальный	ANOVA / Краскел-Уоллис, η^2	`scipy.stats`
любой – цель	mutual information (ловит нелинейность)	`sklearn.feature_selection`

6.1 Корреляции числовых признаков

Код 6.1 — матрица корреляций и сильные пары PYTHON

```python
corr = df[num_cols + [TARGET]].corr(method="spearman")
mask = np.triu(np.ones_like(corr, dtype=bool))
plt.figure(figsize=(8, 6))
sns.heatmap(corr, mask=mask, annot=True, fmt=".2f", cmap="coolwarm",
            center=0, vmin=-1, vmax=1, square=True)
plt.title("Корреляция Спирмена")
plt.show()

# Пары с сильной корреляцией (кандидаты на удаление одного из признаков)
c_abs = df[num_cols].corr().abs()
upper = c_abs.where(np.triu(np.ones(c_abs.shape, dtype=bool), k=1))
pairs = upper.stack().sort_values(ascending=False)
print(pairs[pairs > 0.8])
```

6.2 Категориальные признаки: Cramér's V

Код 6.2 — ассоциации категорий PYTHON

```python
def cramers_v(x, y):
    ct = pd.crosstab(x, y)
    chi2 = stats.chi2_contingency(ct)[0]
    n = ct.to_numpy().sum()
    r, k = ct.shape
    phi2 = max(0, chi2 / n - (k - 1) * (r - 1) / (n - 1))   # поправка на смещение
    r_c = r - (r - 1) ** 2 / (n - 1)
    k_c = k - (k - 1) ** 2 / (n - 1)
    return float(np.sqrt(phi2 / max(1e-12, min(k_c - 1, r_c - 1))))


cv = pd.DataFrame(index=cat_cols, columns=cat_cols, dtype=float)
for a in cat_cols:
    for b in cat_cols:
```

```
        cv.loc[a, b] = 1.0 if a == b else cramers_v(df[a], df[b])

sns.heatmap(cv, annot=True, fmt=".2f", cmap="Blues", vmin=0, vmax=1)
plt.title("Cramér's V")
plt.show()
```

6.3 Связь признаков с целевой переменной

Код 6.3 — mutual information, графики, ставки по категориям PYTHON

```python
from sklearn.feature_selection import mutual_info_classif
from sklearn.preprocessing import OrdinalEncoder

# Числовые признаки
X_num = df[num_cols].fillna(df[num_cols].median())
mi_num = pd.Series(
    mutual_info_classif(X_num, df[TARGET], random_state=SEED), index=num_cols)

# Категориальные признаки (порядковое кодирование + discrete_features)
X_cat = OrdinalEncoder().fit_transform(df[cat_cols].astype("string").fillna("NA"))
mi_cat = pd.Series(
    mutual_info_classif(X_cat, df[TARGET], discrete_features=True, random_state=SEED),
    index=cat_cols)

mi = pd.concat([mi_num, mi_cat]).sort_values(ascending=False)
mi.plot.barh(figsize=(6, 0.35 * len(mi) + 1)).invert_yaxis()
plt.title("Mutual information с целью")
plt.show()

# Топ числовых признаков: распределение по классам
top = mi_num.sort_values(ascending=False).head(4).index
fig, axes = plt.subplots(1, len(top), figsize=(4 * len(top), 3.5))
for ax, c in zip(np.atleast_1d(axes), top):
    sns.boxplot(data=df, x=TARGET, y=c, orient="v", ax=ax)
plt.tight_layout()
plt.show()

# Категориальные: средняя цель по значениям
for c in cat_cols:
    tbl = df.groupby(c)[TARGET].agg(["mean", "count"]).sort_values("mean")
    print(f"\n{c}\n{tbl.round(3)}")
```

6.4 Мультиколлинеарность: VIF

VIF показывает, насколько признак линейно выражается через остальные. Ориентиры: до 5 — нормально, 5–10 — внимание, больше 10 — серьёзная коллинеарность. Для линейных моделей это критично, для деревьев почти безразлично.

Код 6.4 — VIF на чистом scikit-learn PYTHON

```python
from sklearn.linear_model import LinearRegression


def vif_table(X):
    X = X.dropna()
    out = {}
    for c in X.columns:
        others = X.drop(columns=c)
        r2 = LinearRegression().fit(others, X[c]).score(others, X[c])
        out[c] = 1 / (1 - r2) if r2 < 1 else np.inf
    return pd.Series(out, name="VIF").sort_values(ascending=False)


print(vif_table(df[num_cols]).round(2))
# Аналог из statsmodels:
#   from statsmodels.stats.outliers_influence import variance_inflation_factor
```

6.5 Парные графики

Код 6.5 — pairplot по важным признакам PYTHON

```python
cols_pp = mi_num.sort_values(ascending=False).head(4).index.tolist()
sample = df[cols_pp + [TARGET]].sample(min(len(df), 1500), random_state=SEED)
sns.pairplot(sample, hue=TARGET, corner=True, plot_kws={"alpha": 0.5, "s": 15})
plt.show()
```

СОВЕТ

Высокая корреляция не означает причинность и не гарантирует пользы признака. Ищите также **нелинейные** связи (MI, графики) и **взаимодействия**: признаки, полезные только вместе.

7 Распределения и гипотезы

Нормальность, сравнение групп, поправка на множественные тесты

7.1 Проверка нормальности

Код 7.1 — Shapiro-Wilk, QQ-plot, сводка PYTHON

```python
s = df["income"].dropna()
sample = s.sample(min(5000, len(s)), random_state=SEED)     # Shapiro: до ~5000
w, pval = stats.shapiro(sample)
print(f"Shapiro-Wilk: W={w:.3f}, p={pval:.2e}")

fig, ax = plt.subplots(1, 2, figsize=(10, 4))
stats.probplot(sample, dist="norm", plot=ax[0])
ax[0].set_title("QQ-plot: исходный")
stats.probplot(np.log1p(sample.clip(lower=0)), dist="norm", plot=ax[1])
ax[1].set_title("QQ-plot: log1p")
plt.tight_layout()
plt.show()

# Сводка по всем числовым признакам
rows = []
for c in num_cols:
    x = df[c].dropna()
    rows.append({"feature": c, "skew": x.skew(), "normaltest_p": stats.normaltest(x)[1]})
print(pd.DataFrame(rows).set_index("feature").round(4))
```

ВНИМАНИЕ

На больших выборках любой тест нормальности отвергает нормальность даже при незначительных отклонениях. Опирайтесь на QQ-plot и асимметрию, а не только на p-value.

7.2 Сравнение групп и связь с целью

Задача	Параметрический тест	Непараметрический
2 группы (числовой признак)	t-тест Уэлча `ttest_ind(equal_var=False)`	Манна-Уитни `mannwhitneyu`
3+ групп (числовой признак)	ANOVA `f_oneway`	Краскел-Уоллис `kruskal`
категория vs цель	—	χ^2 `chi2_contingency`

Код 7.2 — тесты и размер эффекта PYTHON

```python
def cohens_d(a, b):
    na, nb = len(a), len(b)
    pooled = np.sqrt(((na - 1) * a.var(ddof=1) + (nb - 1) * b.var(ddof=1)) / (na + nb - 2))
    return (a.mean() - b.mean()) / pooled
```

```python
rows = []
for c in num_cols:
    g0 = df.loc[df[TARGET] == 0, c].dropna()
    g1 = df.loc[df[TARGET] == 1, c].dropna()
    rows.append({
        "feature": c,
        "median_0": g0.median(), "median_1": g1.median(),
        "mannwhitney_p": stats.mannwhitneyu(g0, g1).pvalue,
        "cohens_d": cohens_d(g1, g0),
    })
num_tests = pd.DataFrame(rows).set_index("feature")
display(num_tests.round(4))

rows = []
for c in cat_cols:
    chi2, pval, dof, _ = stats.chi2_contingency(pd.crosstab(df[c], df[TARGET]))
    rows.append({"feature": c, "chi2": chi2, "p_value": pval, "dof": dof})
cat_tests = pd.DataFrame(rows).set_index("feature")
display(cat_tests.round(4))
```

СОВЕТ

p-value отвечает на вопрос «есть ли эффект», но не «насколько он велик». Всегда смотрите на размер эффекта: |d| около 0.2 малый, 0.5 средний, 0.8 большой.

7.3 Множественные сравнения

Проверяя 50 признаков при уровне значимости 5 %, мы в среднем получим 2–3 «значимых» случайно. Поправка Бенджамини-Хохберга (FDR) контролирует долю ложных открытий.

Код 7.3 — поправка Бенджамини-Хохберга PYTHON

```python
pvals = pd.concat([num_tests["mannwhitney_p"], cat_tests["p_value"]])
adj = pd.Series(stats.false_discovery_control(pvals.to_numpy(), method="bh"),
                index=pvals.index)
result = pd.DataFrame({"p": pvals, "p_adj_BH": adj}).sort_values("p")
result["significant_5%"] = result["p_adj_BH"] < 0.05
print(result.round(4))
```

8 Проверка на утечки данных

Когда модель «знает» то, чего знать не должна

Утечка (data leakage) даёт нереалистично хорошие метрики на валидации и провал в продакшене. Типичные источники: признак, вычисленный после события; прямой потомок цели; пересечение объектов между train и test; неправильный временной порядок.

8.1 Подозрительно сильные признаки

Код 8.1 — модель на одном признаке PYTHON

```python
from sklearn.model_selection import cross_val_score
from sklearn.tree import DecisionTreeClassifier

# 1) Поиск по названиям
SUSPECT = ("target", "label", "result", "outcome", "status", "after", "future", "churn")
by_name = [c for c in df.columns if c != TARGET and any(w in c.lower() for w in SUSPECT)]
print("Подозрительные названия:", by_name)

# 2) Одиночная модель на каждом признаке: AUC > 0.9 — повод разобраться
single_auc = {}
for c in num_cols:
    X1 = df[[c]].fillna(df[c].median())
    tree = DecisionTreeClassifier(max_depth=3, random_state=SEED)
    single_auc[c] = cross_val_score(tree, X1, df[TARGET], cv=5, scoring="roc_auc").mean()
print(pd.Series(single_auc).sort_values(ascending=False).round(3))

# 3) ID и порядок строк не должны коррелировать с целью
print(df[ID_COLS + [TARGET]].corr(method="spearman").round(3))
```

СОВЕТ

Для каждого признака задайте вопрос: **«будет ли это значение известно в момент, когда модель делает предсказание?»** Если нет, это утечка, какой бы красивой ни была метрика.

8.2 Пересечения train / test

Код 8.2 — дубликаты и общие объекты между выборками PYTHON

```python
feat_cols = [c for c in train.columns if c not in [TARGET] + ID_COLS]

# Полностью совпадающие строки в train и test
dups = train[feat_cols].merge(test[feat_cols].drop_duplicates(), how="inner")
print("Строк train, совпадающих со строками test:", len(dups))

# Общие идентификаторы (если один объект встречается несколько раз)
shared = set(train["user_id"]) & set(test["user_id"])
print(f"Общих user_id: {len(shared)} "
      f"({len(shared) / test['user_id'].nunique():.1%} от test)")
```

ЗАМЕТКА
Если один и тот же пользователь (устройство, пациент) есть и в train, и в test, модель может «запоминать» его. Используйте **GroupKFold** и делите по группам, а не по строкам.

8.3 Корректность временного разбиения

Код 8.3 — проверка и правильный split по времени PYTHON

```python
print("train:", train["date"].min(), "->", train["date"].max())
print("test: ", test["date"].min(), "->", test["date"].max())
if train["date"].max() > test["date"].min():
    print("WARNING: train и test пересекаются по времени")

# Правильный split для прогнозных задач
cutoff = df_raw["date"].quantile(0.8)
train_t = df_raw[df_raw["date"] <= cutoff]
test_t = df_raw[df_raw["date"] > cutoff]
print(f"split по дате {cutoff:%Y-%m-%d}: {len(train_t)} / {len(test_t)}")
```

9 Репрезентативность и смещения

Похожи ли данные на то, что увидит модель в жизни

9.1 Сравнение распределений train и test: KS-тест и PSI

PSI (Population Stability Index) интерпретируется так: менее 0.1 — различий нет; 0.1–0.25 — умеренный сдвиг; более 0.25 — сильный сдвиг, модель, обученная на старых данных, скорее всего деградирует.

Код 9.1 — KS-тест и PSI по всем признакам PYTHON

```python
def psi(expected, actual, bins=10):
    expected, actual = expected.dropna(), actual.dropna()
    edges = np.unique(np.quantile(expected, np.linspace(0, 1, bins + 1)))
    edges[0], edges[-1] = -np.inf, np.inf
    e = np.histogram(expected, edges)[0] / len(expected)
    a = np.histogram(actual, edges)[0] / len(actual)
    e, a = np.clip(e, 1e-4, None), np.clip(a, 1e-4, None)
    return float(np.sum((a - e) * np.log(a / e)))


rows = []
for c in num_cols:
    ks = stats.ks_2samp(train[c].dropna(), test[c].dropna())
    rows.append({"feature": c, "ks_stat": ks.statistic, "ks_p": ks.pvalue,
                 "psi": psi(train[c], test[c])})
drift = pd.DataFrame(rows).set_index("feature").sort_values("psi", ascending=False)
display(drift.round(4))
```

9.2 Adversarial validation

Обучаем классификатор отличать train от test. Если ROC-AUC около 0.5, выборки неотличимы; если около 1, они сильно различаются, а важности признаков показывают, чем именно.

Код 9.2 — классификатор «train против test» PYTHON

```python
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_val_predict
from sklearn.metrics import roc_auc_score

adv = pd.concat([train[num_cols].assign(is_test=0),
                 test[num_cols].assign(is_test=1)], ignore_index=True)
X_adv = adv[num_cols].fillna(adv[num_cols].median())
y_adv = adv["is_test"]

rf = RandomForestClassifier(n_estimators=200, max_depth=5, n_jobs=-1, random_state=SEED)
proba = cross_val_predict(rf, X_adv, y_adv, cv=5, method="predict_proba")[:, 1]
print("Adversarial ROC-AUC:", round(roc_auc_score(y_adv, proba), 3))

rf.fit(X_adv, y_adv)
print(pd.Series(rf.feature_importances_, index=num_cols).sort_values(ascending=False))
```

9.3 Дрейф во времени

Код 9.3 — динамика признаков и цели по месяцам PYTHON

```python
monthly = (
    df.sort_values("date").set_index("date")
    .resample("MS")
    .agg({TARGET: "mean", "income": "median", "age": "mean"})
)
monthly.plot(subplots=True, figsize=(10, 6), marker="o", title="Дрейф по месяцам")
plt.tight_layout()
plt.show()
```

9.4 Группы, смещения и чувствительные признаки

Код 9.4 — представленность групп и ставки по группам PYTHON

```python
SENSITIVE = ["gender"]          # пол, возраст, регион и т.п. — по вашей задаче

for c in SENSITIVE:
    tbl = df.groupby(c).agg(n=(TARGET, "size"), target_rate=(TARGET, "mean"))
    tbl["share"] = tbl["n"] / tbl["n"].sum()
    print(f"\n{c}\n{tbl.round(3)}")

# Совпадают ли доли категорий в train и test?
for c in cat_cols:
    cmp = pd.concat([train[c].value_counts(normalize=True),
                     test[c].value_counts(normalize=True)],
                    axis=1, keys=["train", "test"]).fillna(0)
    cmp["abs_diff"] = (cmp["train"] - cmp["test"]).abs()
    print(f"\n{c}\n{cmp.sort_values('abs_diff', ascending=False).head(3).round(3)}")
```

ВНИМАНИЕ
Если модель будет влиять на людей (кредиты, найм, медицина), проверьте качество и долю положительных решений **по группам**. Удаление чувствительного признака не решает проблему: его могут восстанавливать коррелирующие признаки (прокси).

10 Идеи по подготовке данных

Превращаем выводы EDA в пайплайн и стратегию валидации

10.1 Пайплайн предобработки

Все преобразования оформляем в `Pipeline` / `ColumnTransformer`: они обучаются только на обучающей части каждого фолда, и утечка из импутации и масштабирования исключена.

Ситуация	Подход
пропуски < 5 %, MCAR	медиана / мода, либо удаление строк
пропуски информативны	импутация + индикатор `add_indicator=True`
категории, мало значений	OneHotEncoder
категории, много значений	частотное кодирование, target encoding внутри CV, CatBoost
скошенный числовой признак	log1p / PowerTransformer

Код 10.1 — ColumnTransformer PYTHON

```python
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder, PowerTransformer

drop_cols = const_cols + ID_COLS + [TARGET]
use_num = [c for c in num_cols if c not in drop_cols]
use_cat = [c for c in cat_cols if c not in drop_cols]

num_pipe = Pipeline([
    ("impute", SimpleImputer(strategy="median", add_indicator=True)),
    ("power", PowerTransformer()),                # Yeo-Johnson, + стандартизация
])
cat_pipe = Pipeline([
    ("impute", SimpleImputer(strategy="most_frequent")),
    ("ohe", OneHotEncoder(handle_unknown="ignore", min_frequency=0.01)),
])
preprocess = ColumnTransformer([
    ("num", num_pipe, use_num),
    ("cat", cat_pipe, use_cat),
])
print(preprocess)
```

10.2 Идеи для новых признаков

Код 10.2 — календарные, отношения, индикаторы, агрегаты PYTHON

```python
# Календарные
df["month"] = df["date"].dt.month
df["dow"] = df["date"].dt.dayofweek
df["is_weekend"] = (df["dow"] >= 5).astype(int)

# Отношения и преобразования
df["income_per_age"] = df["income"] / df["age"].where(df["age"] > 0)
df["income_missing"] = df["income"].isna().astype(int)

# Агрегаты по группе. Считаем по train, потом ПРИМЕНЯЕМ к test
user_stats = df.groupby("user_id")["income"].agg(["mean", "count"]).add_prefix("user_income_")
df = df.merge(user_stats, left_on="user_id", right_index=True, how="left")
test_with_stats = test.merge(user_stats, left_on="user_id", right_index=True, how="left")
print(df.filter(like="user_income_").describe().round(2))
```

ВНИМАНИЕ

Агрегаты **по целевой переменной** (средняя цель по категории) считайте только внутри кросс-валидации, иначе получите утечку. Для этого используйте `TargetEncoder` из scikit-learn или аналог.

10.3 Валидация и быстрый бейзлайн

Схема	Когда использовать
`StratifiedKFold`	классификация, независимые объекты
`KFold`	регрессия, независимые объекты
`GroupKFold`	несколько строк на пользователя / устройство / пациента
`TimeSeriesSplit`	прогноз будущего, важен порядок во времени

Код 10.3 — бейзлайн как точка отсчёта PYTHON

```python
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import StratifiedKFold, GroupKFold, TimeSeriesSplit

cv_strat = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED)
cv_group = GroupKFold(n_splits=5)       # .split(X, y, groups=df["user_id"])
cv_time = TimeSeriesSplit(n_splits=5)   # данные должны быть отсортированы по времени

baseline = Pipeline([
    ("prep", preprocess),
    ("model", LogisticRegression(max_iter=1000)),
])
X, y = df.drop(columns=[TARGET]), df[TARGET]
auc = cross_val_score(baseline, X, y, cv=cv_strat, scoring="roc_auc")
print(f"Baseline ROC-AUC: {auc.mean():.3f} +/- {auc.std():.3f}")
```

СОВЕТ
Простой бейзлайн нужен как точка отсчёта: любая сложная модель обязана его обыграть. Если не обыгрывает, проблема в данных или признаках, а не в алгоритме.

11 Документирование результатов

Чтобы выводы не потерялись, а анализ можно было повторить

11.1 Автоматические отчёты

Автоотчёты хороши как **стартовая точка**: они за минуты показывают основные проблемы. Но они не заменяют вдумчивый анализ и не знают вашу предметную область.

Код 11.1 — ydata-profiling и sweetviz PYTHON

```python
# pip install ydata-profiling sweetviz
from ydata_profiling import ProfileReport

ProfileReport(df, title="EDA: train", minimal=True).to_file("eda_report.html")

# Сравнение train и test в одном отчёте
import sweetviz as sv
sv.compare([train, "train"], [test, "test"], target_feat=TARGET).show_html("compare.html")
```

11.2 Структура итогового отчёта

- **Резюме**: 3–5 главных находок и их влияние на моделирование.
- **Данные**: источник, период, размер, словарь признаков.
- **Качество**: таблица проблем (пропуски, дубликаты, ошибки) и принятое решение по каждой.
- **Цель**: распределение, баланс, выбранная метрика.
- **Ключевые закономерности**: 3–7 графиков с подписями-выводами.
- **Риски**: утечки, дрейф, смещения, ограничения данных.
- **План**: признаки для создания, схема валидации, гипотезы для проверки моделью.

11.3 Воспроизводимость

Код 11.3 — сохраняем решения и окружение PYTHON

```python
import json, platform, sklearn
from pathlib import Path

Path("artifacts").mkdir(exist_ok=True)

decisions = {
    "seed": SEED,
    "split": {"test_size": 0.2, "stratify": TARGET},
    "drop_cols": const_cols + ID_COLS,
    "log_transform": ["income"],
    "winsorize": {"income": [float(low), float(high)]},
```

```
    "validation": "StratifiedKFold(5)",
}
env = {"python": platform.python_version(), "pandas": pd.__version__,
       "numpy": np.__version__, "sklearn": sklearn.__version__}

Path("artifacts/eda_decisions.json").write_text(
    json.dumps({"decisions": decisions, "env": env}, ensure_ascii=False, indent=2),
    encoding="utf-8")
train.to_csv("artifacts/train.csv", index=False)     # или .to_parquet(...)
test.to_csv("artifacts/test.csv", index=False)
```

СОВЕТ

Зафиксируйте `random_state`, версии библиотек (`pip freeze > requirements.txt`) и сохраните конкретные train/test, на которых делались выводы. Через месяц вы скажете себе спасибо.

A Ошибки и чек-лист

Чего избегать и что проверить перед моделированием

A.1 Типичные ошибки

- **Предобработка до split.** Среднее, медиана и границы выбросов, посчитанные на всех данных, «просачиваются» в test.
- **Решения по test.** Если вы смотрели на test и подстраивали признаки, он перестал быть независимой проверкой.
- **Удаление выбросов по умолчанию.** Редкое реальное событие может быть самым ценным наблюдением.
- **Слепая вера в p-value.** На больших выборках значимо всё; при сотне тестов случайно значимы несколько. Смотрите на размер эффекта и поправки.
- **Корреляция = причинность.** Связь в данных не говорит о том, что признак можно «подкрутить» и получить результат.
- **Игнорирование смысла признаков.** Именно так в модель попадают данные из будущего.
- **Случайный split для временных и групповых данных.** Метрики на валидации окажутся оптимистичными.
- **Среднее вместо анализа пропусков.** Механизм пропуска важнее метода заполнения.
- **EDA ради графиков.** Каждый график должен заканчиваться выводом и решением.

A.2 Итоговый чек-лист

☐ Данные прочитаны верно, смысл признаков понятен	☐ Проверены корреляции и VIF
☐ Сделан ранний split, test не тронут	☐ Оценена связь признаков с целью
☐ Определены тип задачи и метрика	☐ Тесты с учётом размера эффекта и поправки
☐ Изучены цель и баланс классов	☐ Проверены утечки: признаки, пересечения, время
☐ Оценены пропуски: доля, механизм, связь с целью	☐ Сравнены train и test (PSI, adversarial)
☐ Найдены дубликаты (полные и по ключу)	☐ Проверены дрейф и смещения по группам
☐ Проверены бизнес-правила и невозможные значения	☐ Выбрана схема валидации
☐ Приведены к порядку категории	☐ Составлен план предобработки и признаков
☐ Найдены константы и идентификаторы	☐ Построен простой бейзлайн
☐ Изучены распределения признаков	☐ Выводы и решения задокументированы
☐ Выбросы диагностированы, решение принято	☐ Зафиксированы seed и версии библиотек